



Encontro 1

Fundamentos e Primeiros Passos

0:00–0:35

Teoria + contexto

0:35–1:00

Instalação guiada

1:00–1:30

GitHub + VSCode

1:30–2:40

Prática comandos

2:40–3:00

Quiz (10 perguntas)

```
git commit -m "Vamos começar!"
```

O que são Git e GitHub?



Git

Software instalado na sua máquina

- Controla versões localmente
- Funciona sem internet
- Guarda o histórico completo
- Criado por Linus Torvalds (2005)
- Software livre e gratuito

≠



GitHub

Plataforma web — rede social de devs

- Hospeda repositórios Git
- Colaboração em equipe
- Portfólio público de projetos
- Fundado em 2008
- Gratuito para uso educacional

Não confunda Git com GitHub

Analogia 1

Git

instalado no seu computador

GitHub

na nuvem, acessível de qualquer lugar

Analogia 2

Git

funciona mesmo sem sinal

GitHub

conecta você com o resto do mundo

Regra de ouro

Você pode usar o Git sem o GitHub (trabalho local). Mas não pode usar o GitHub sem o Git instalado na máquina.

História do Git e do GitHub



Como era antes do Git?

Pasta do projeto do colega

-  projeto/
-  projeto_v2/
-  projeto_v2_certo/
-  projeto_v3_FINAL/
-  projeto_FINAL_ok/
-  projeto_FINAL_ok_agora/
-  projeto_ESSE_FUNCIONA/
-  projeto_ESSE_FUNCIONA_v2/

Os problemas

-  Qual versão estava funcionando?
-  O que mudou entre as versões?
-  Quem fez qual alteração?
-  Como trabalhar junto no mesmo arquivo?
-  E se eu enviar por e-mail e ele editar?

 **Git resolve tudo isso.**



Criação de conta e repositório no GitHub

Criar conta no GitHub

- 1 Acessar github.com e clicar em Sign up
- 2 Usar e-mail pessoal ou institucional
- ! Escolha bem o nome de usuário — ele vai aparecer no currículo e portfólio
- 4 Verificar o e-mail e completar o cadastro
- 5 Explorar feed, perfil e aba de repositórios

Criar o 1º repositório

- 1 Clicar no botão verde "New" no perfil
- 2 Definir o nome: meu-primeiro-projeto
- 3 Visibilidade: Public (aparece no portfólio)
- 4 Marcar "Add a README file"
- 5 Clicar em Create repository e explorar

>_ Instalação do Git e teste no terminal

1

Baixar o instalador

Acessar git-scm.com, clicar em Download for Windows (ou Mac/Linux). Executar o instalador com opções padrão.

2

Verificar a instalação

Abrir o terminal (Git Bash no Windows, Terminal no Mac/Linux) e digitar:

```
$ git --version
```

3

Configurar a identidade

Esses dados aparecem em todos os seus commits — use nome e e-mail profissional:

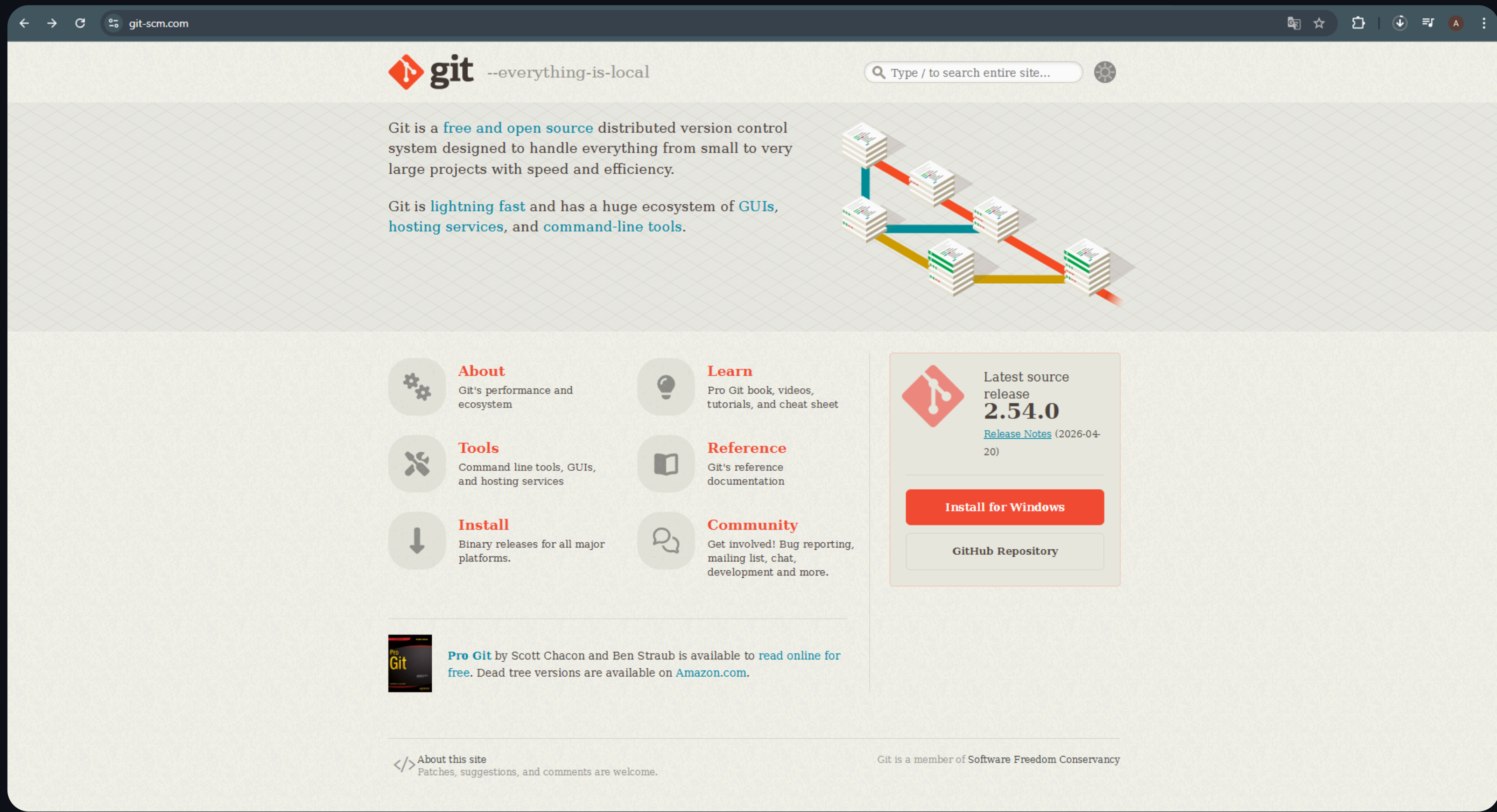
```
$ git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"
```

4

Confirmar a configuração

Listar todas as configurações salvas:

```
$ git config --list
```



git

--distributed-even-if-your-workflow-isnt

Type / to search entire site...

About

Learn

Tools

Reference

Install

Community

The entire [Pro Git book](#) written by Scott Chacon and Ben Straub is available to [read online for free](#). Dead tree versions are available on [Amazon.com](#).

Install

Latest version: 2.54.0 ([Release Notes](#))

Windows

macOS

Linux

Build from Source

[Click here to download](#) the latest (2.54.0) x64 version of Git for Windows. This is the most recent [maintained build](#). It was released **19 days ago**, on 2026-04-20.

Other Git for Windows downloads

Standalone Installer
[Git for Windows/x64 Setup](#).

[Git for Windows/ARM64 Setup](#).

Portable ("thumbdrive edition")
[Git for Windows/x64 Portable](#).

[Git for Windows/ARM64 Portable](#).

Using winget tool
Install [winget tool](#) if you don't already have it, then type this command in command prompt or Powershell.

```
winget install --id Git.Git -e --source winget
```

The current source code release is version 2.54.0. If you want the newer version, you can build it from [the source code](#).

Now What?

Now that you have downloaded Git, it's time to start using it.



Read the Book

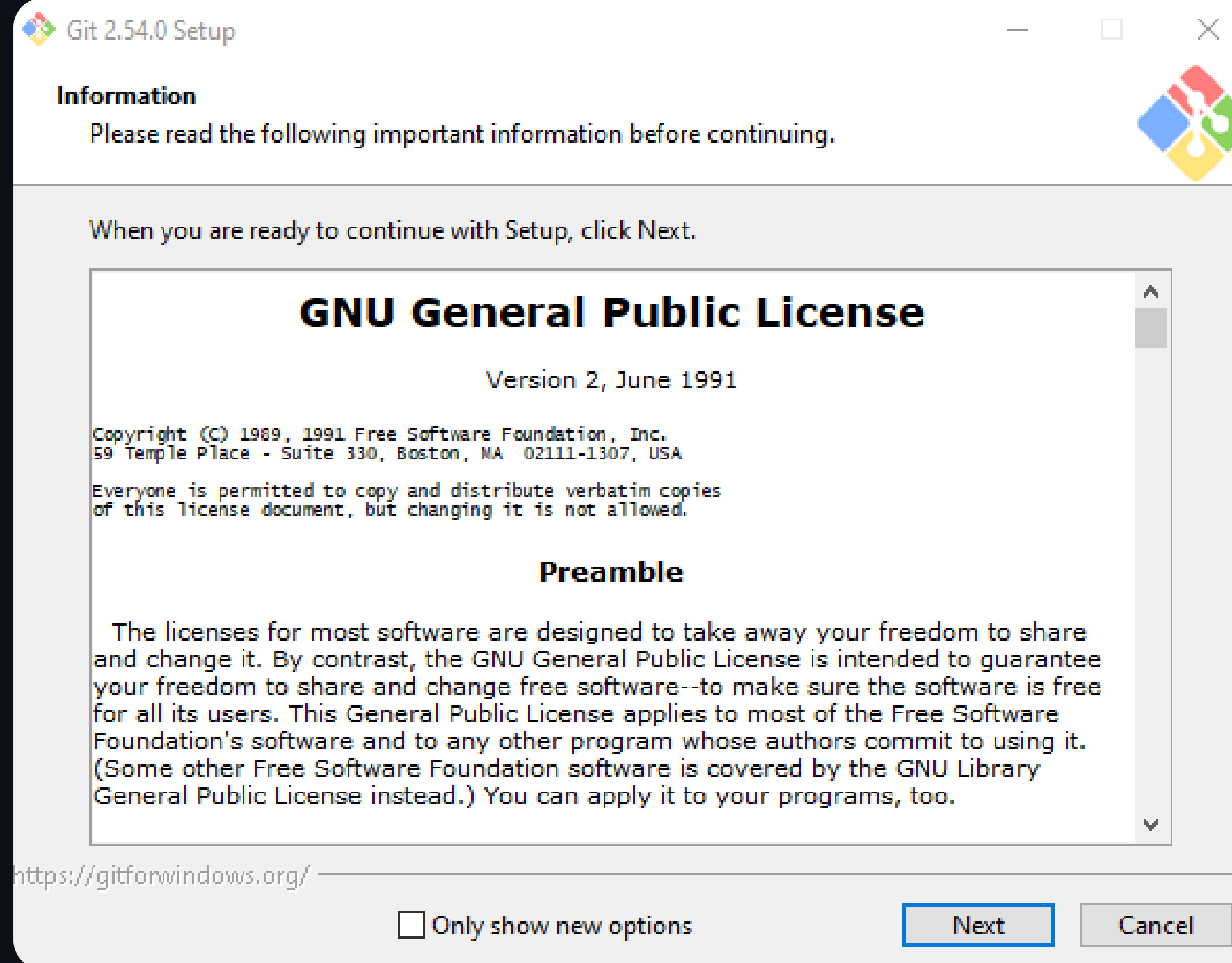
Dive into the Pro Git book and learn at your own pace.

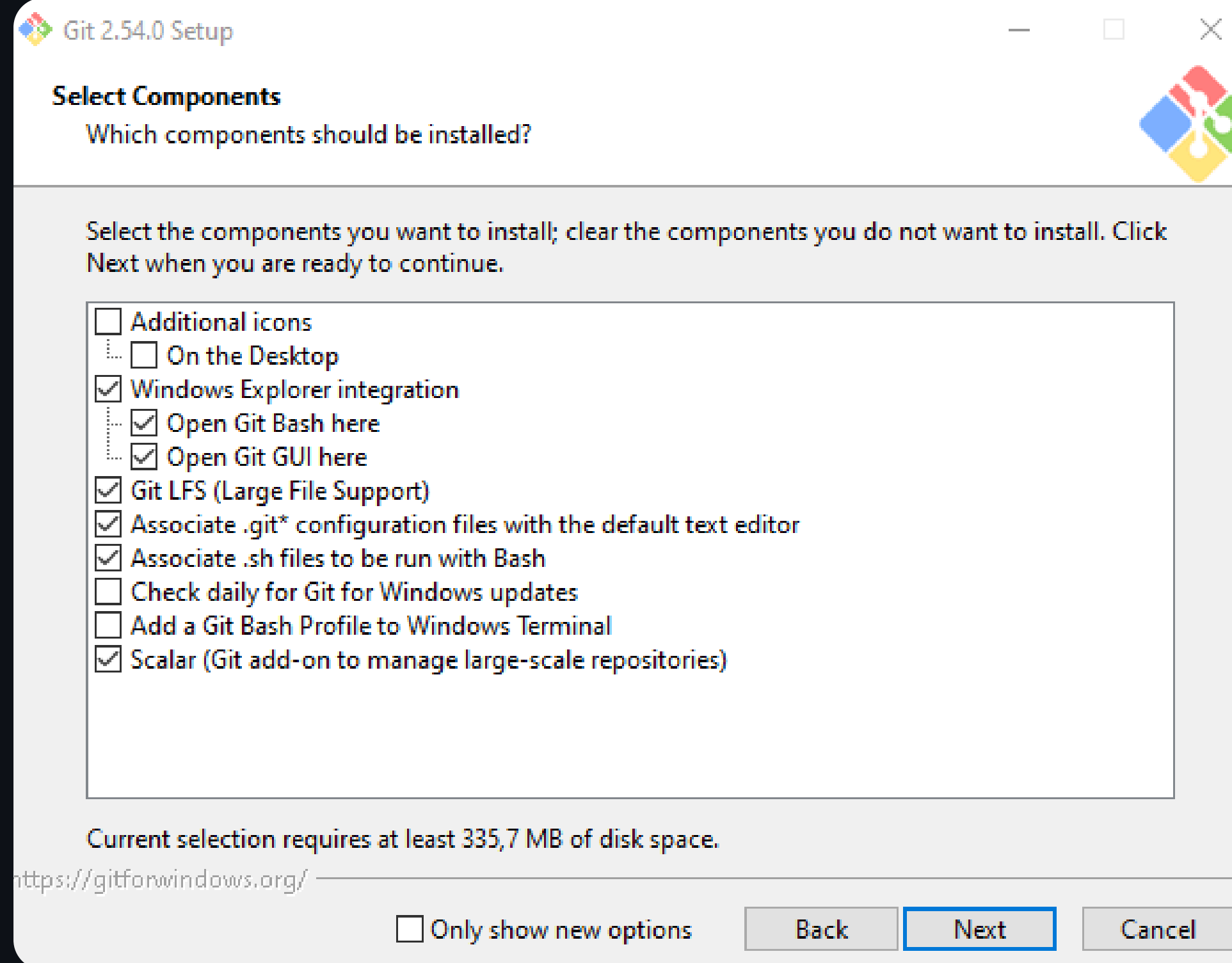


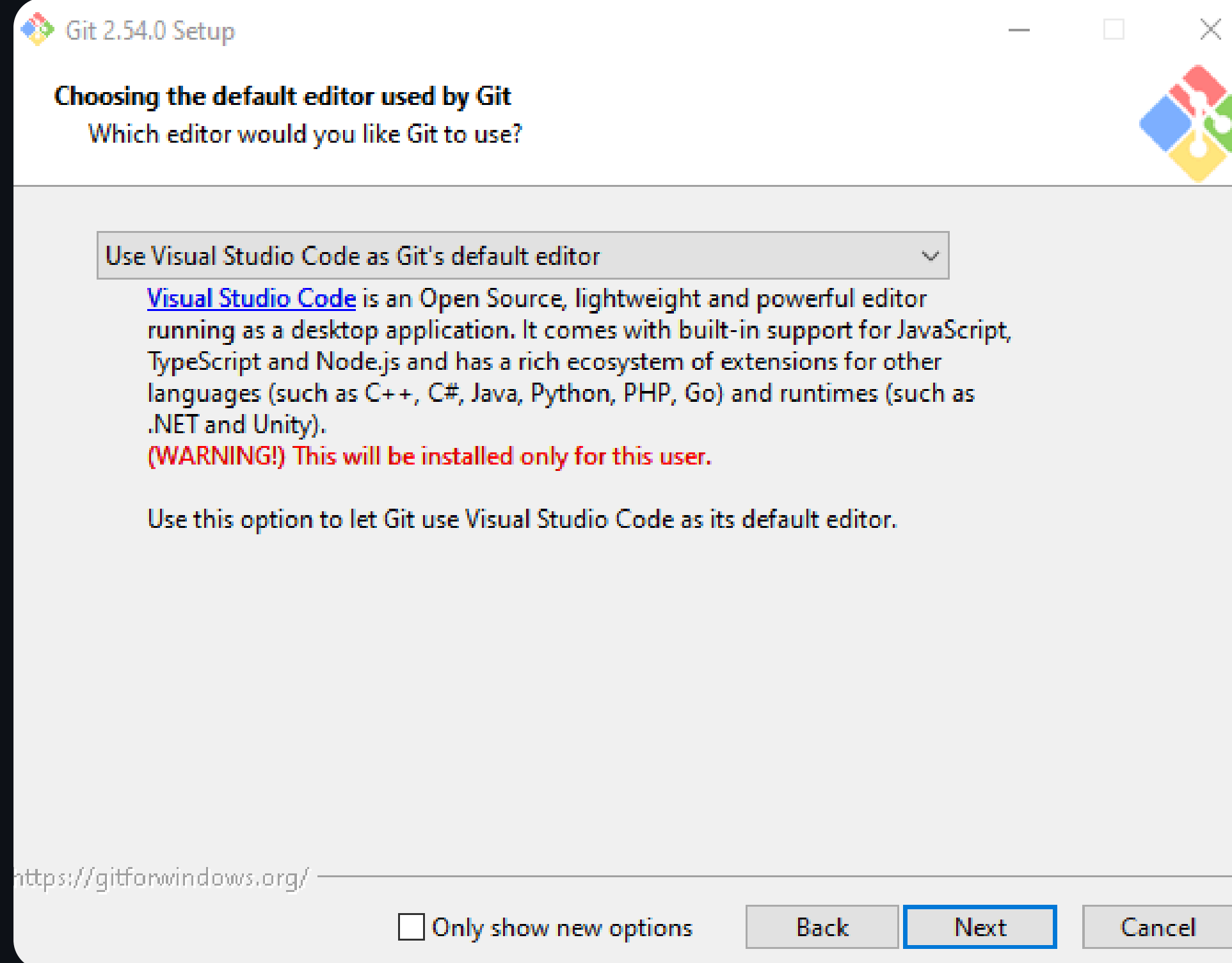
Download a GUI

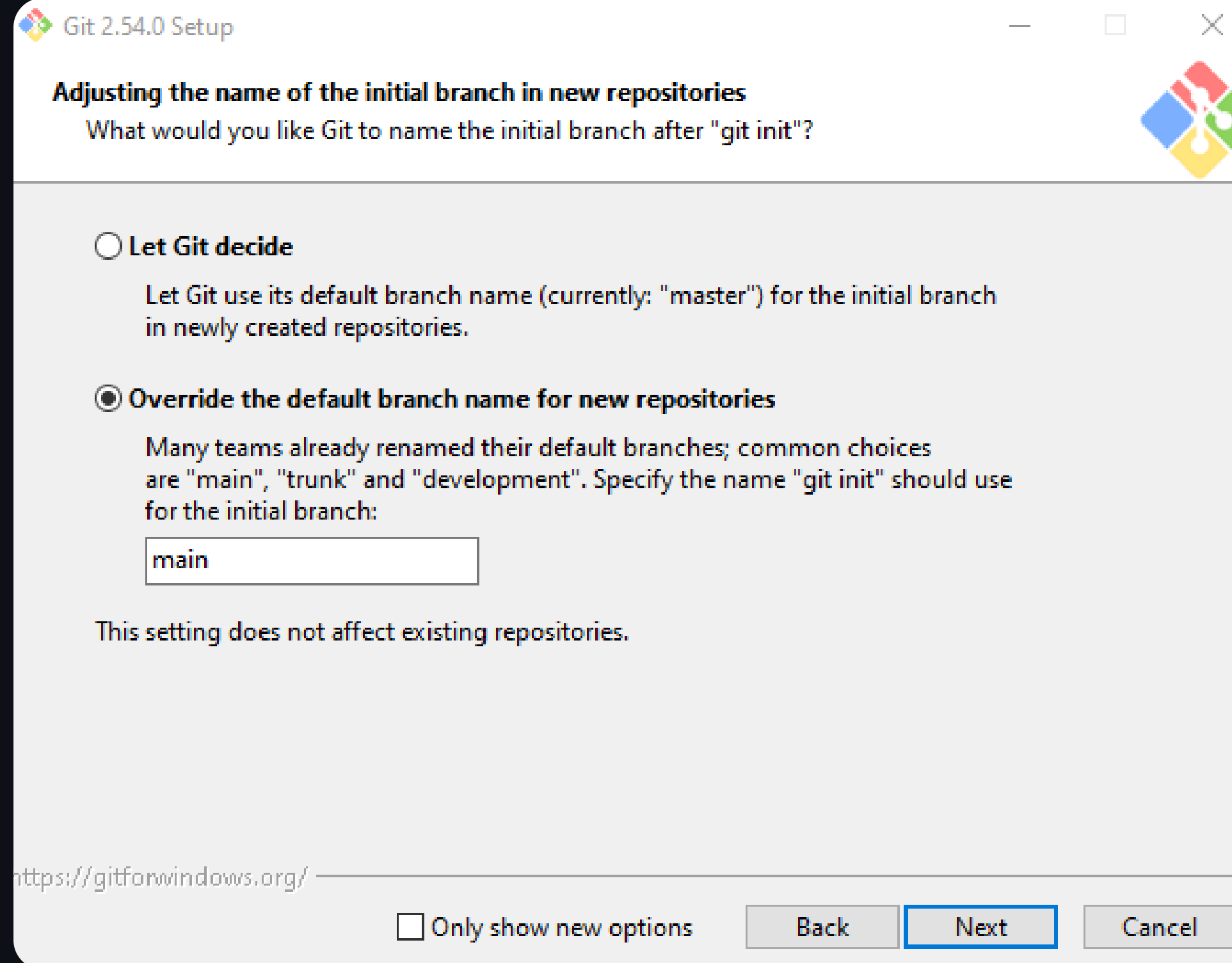
Several free and commercial GUI tools are available for the Windows platform.

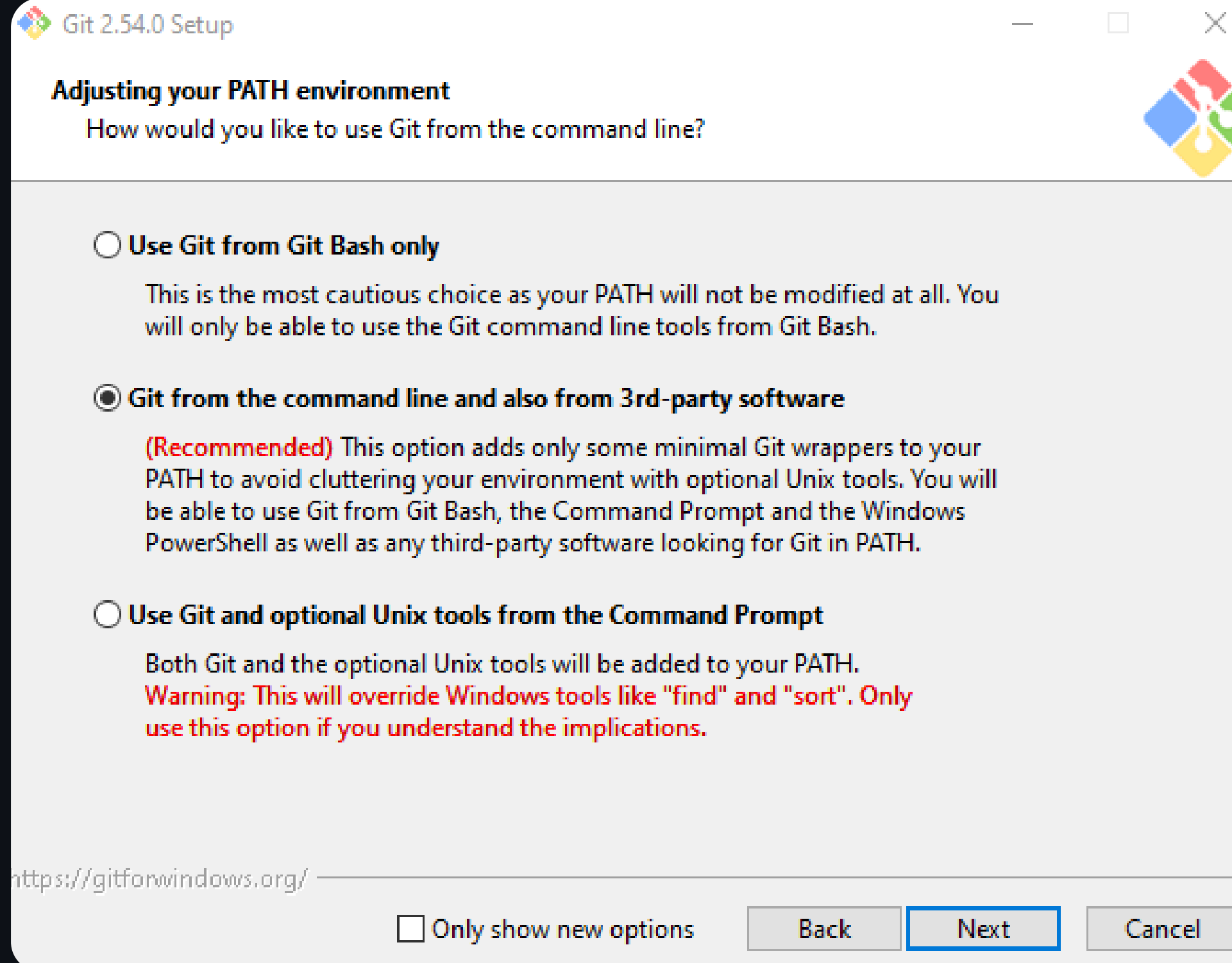
Encontro 1 / Instalação do Git

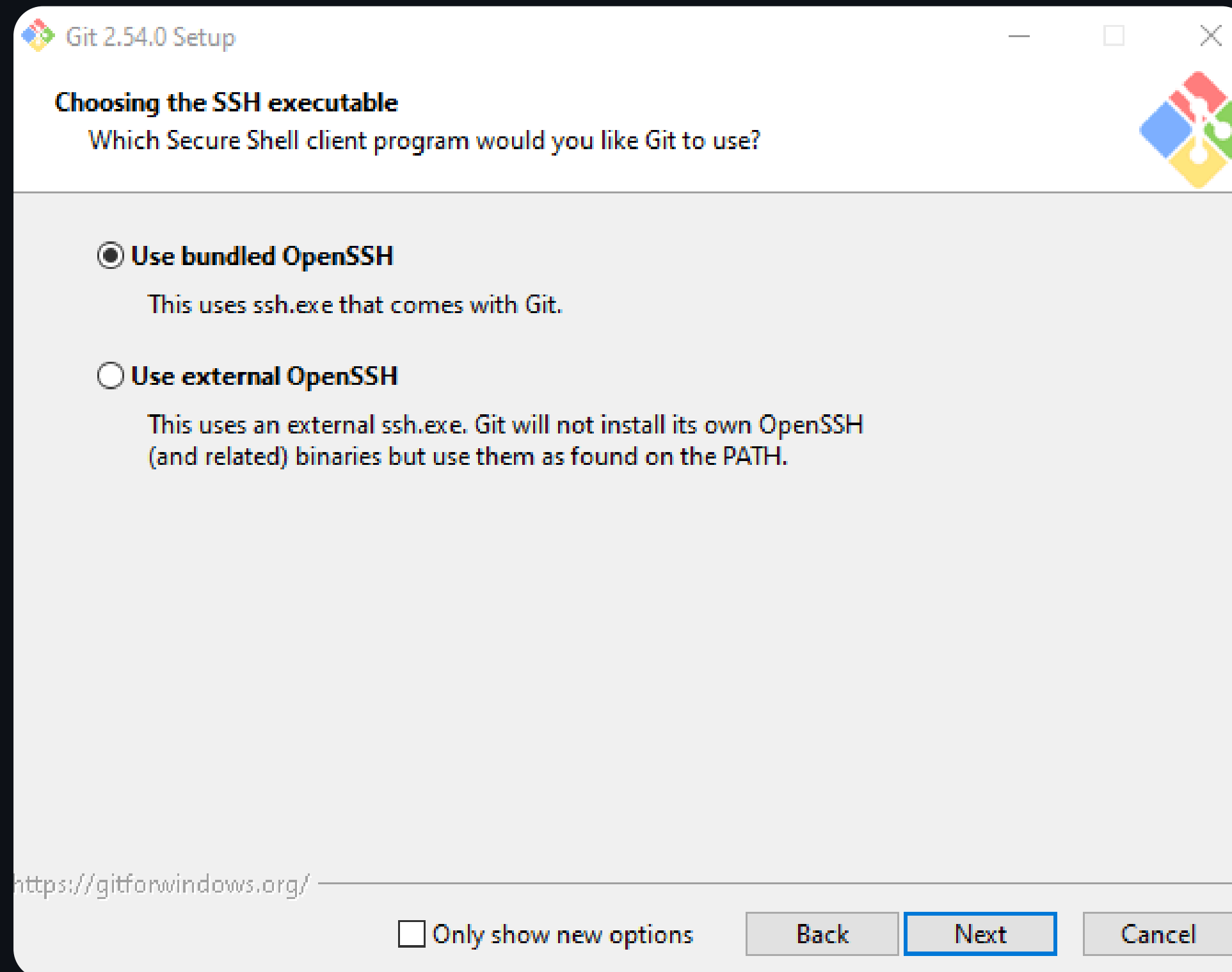


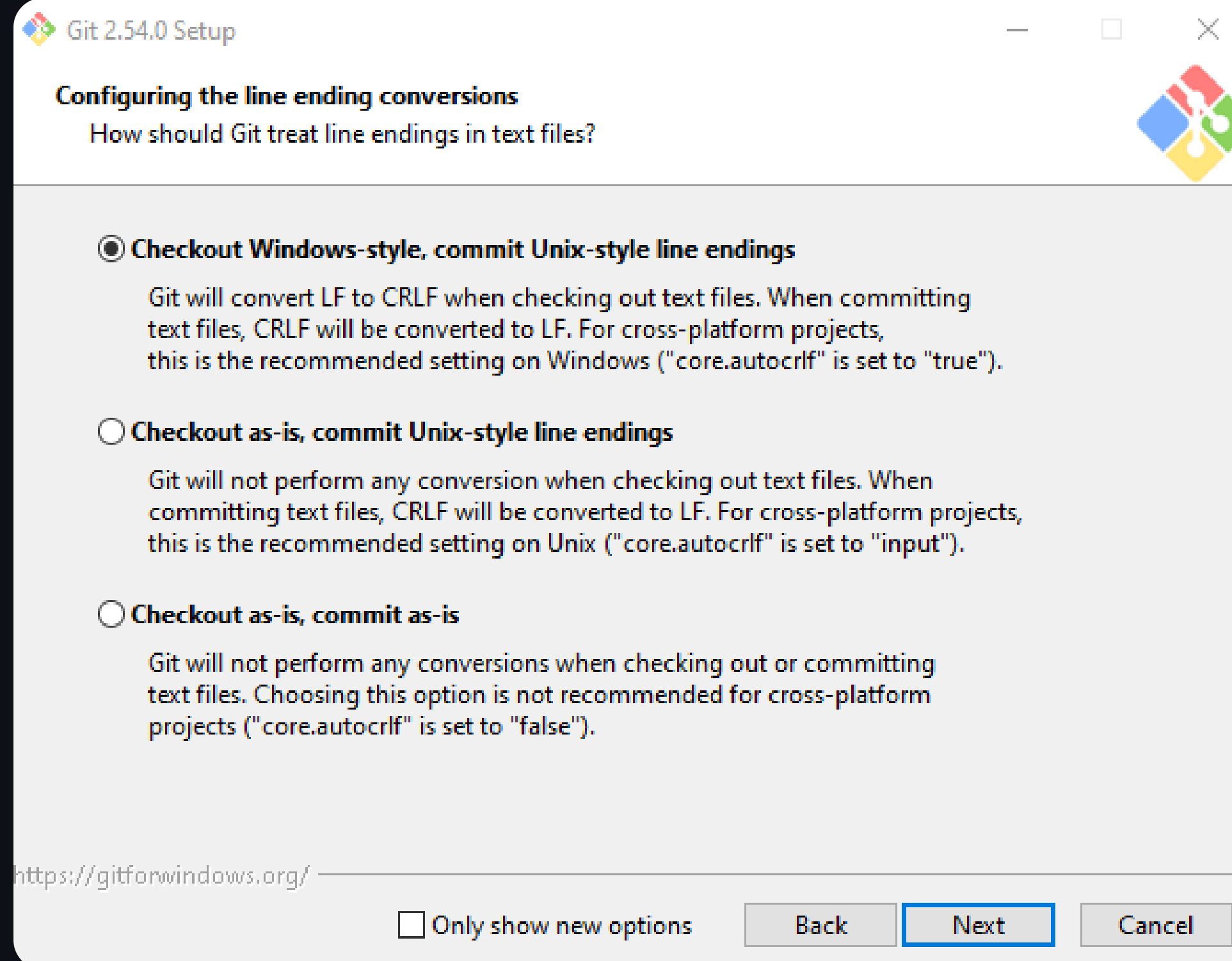


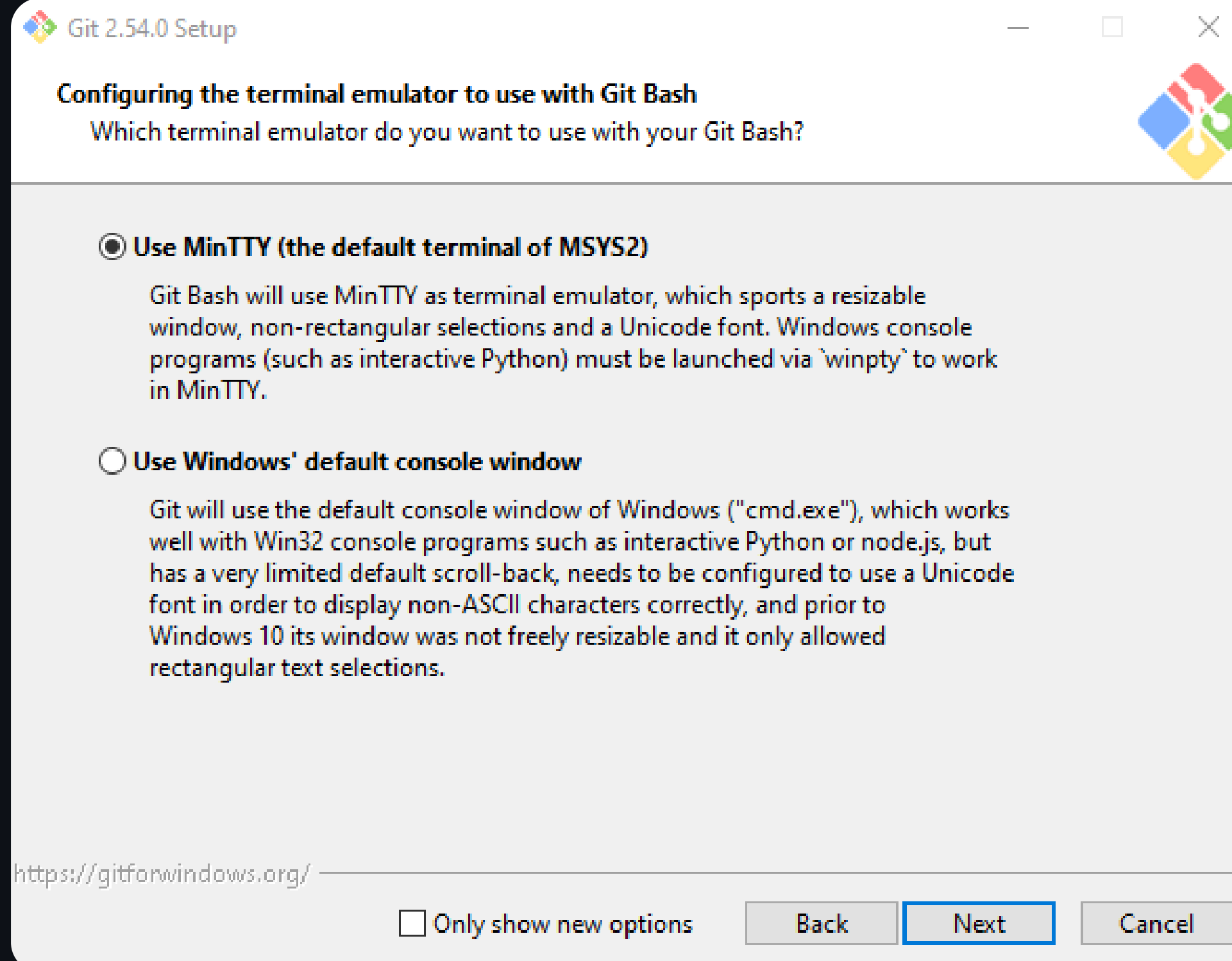


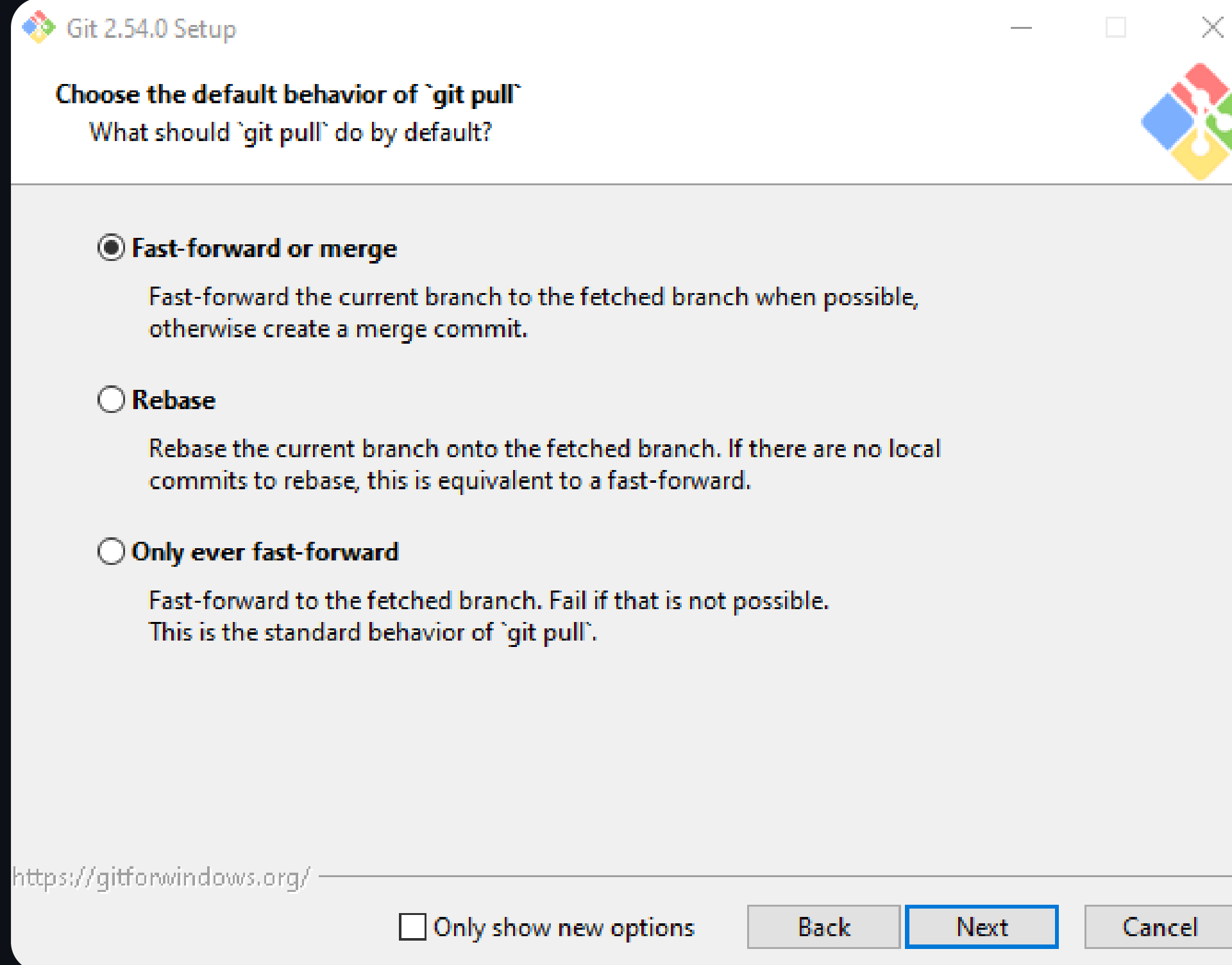


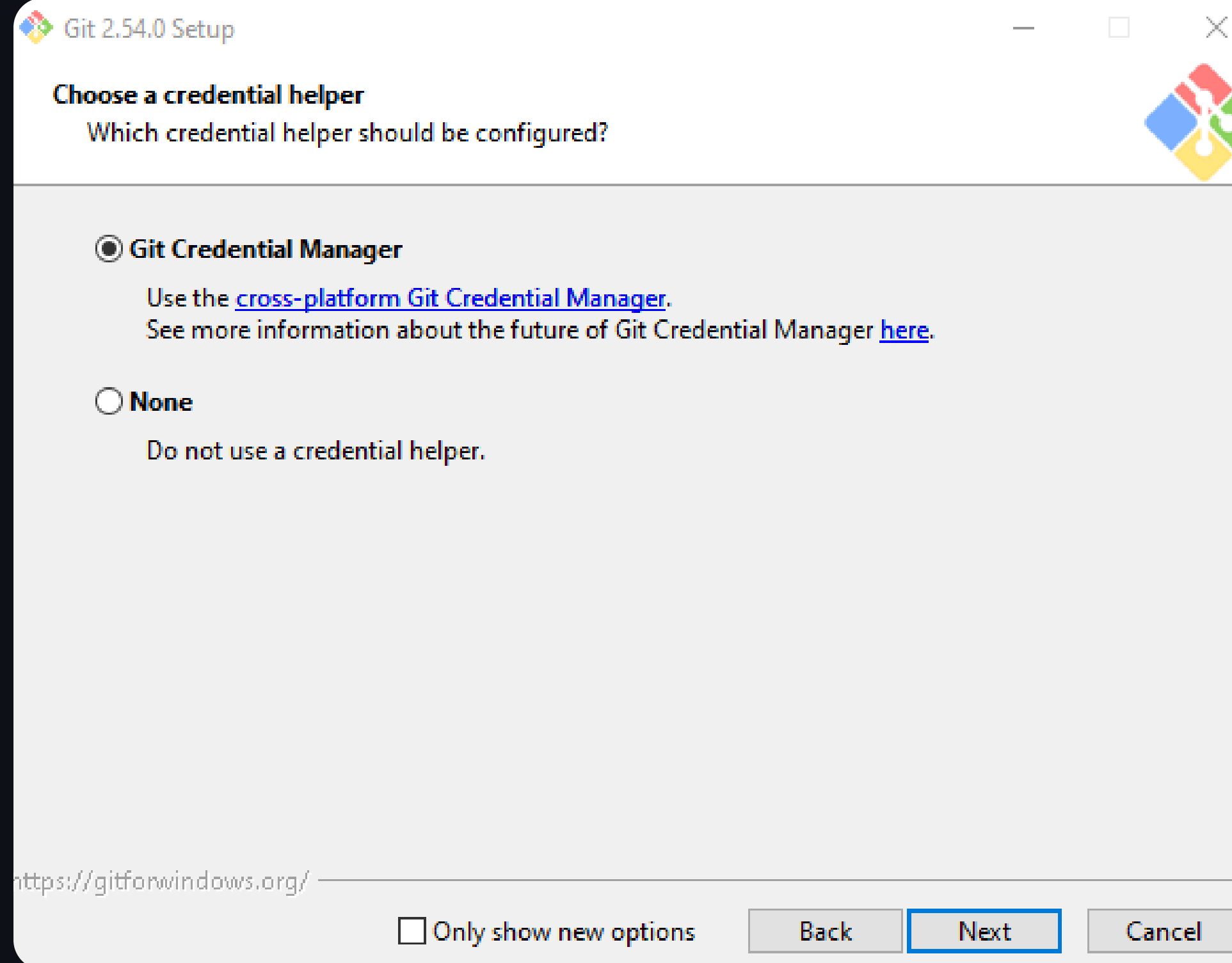


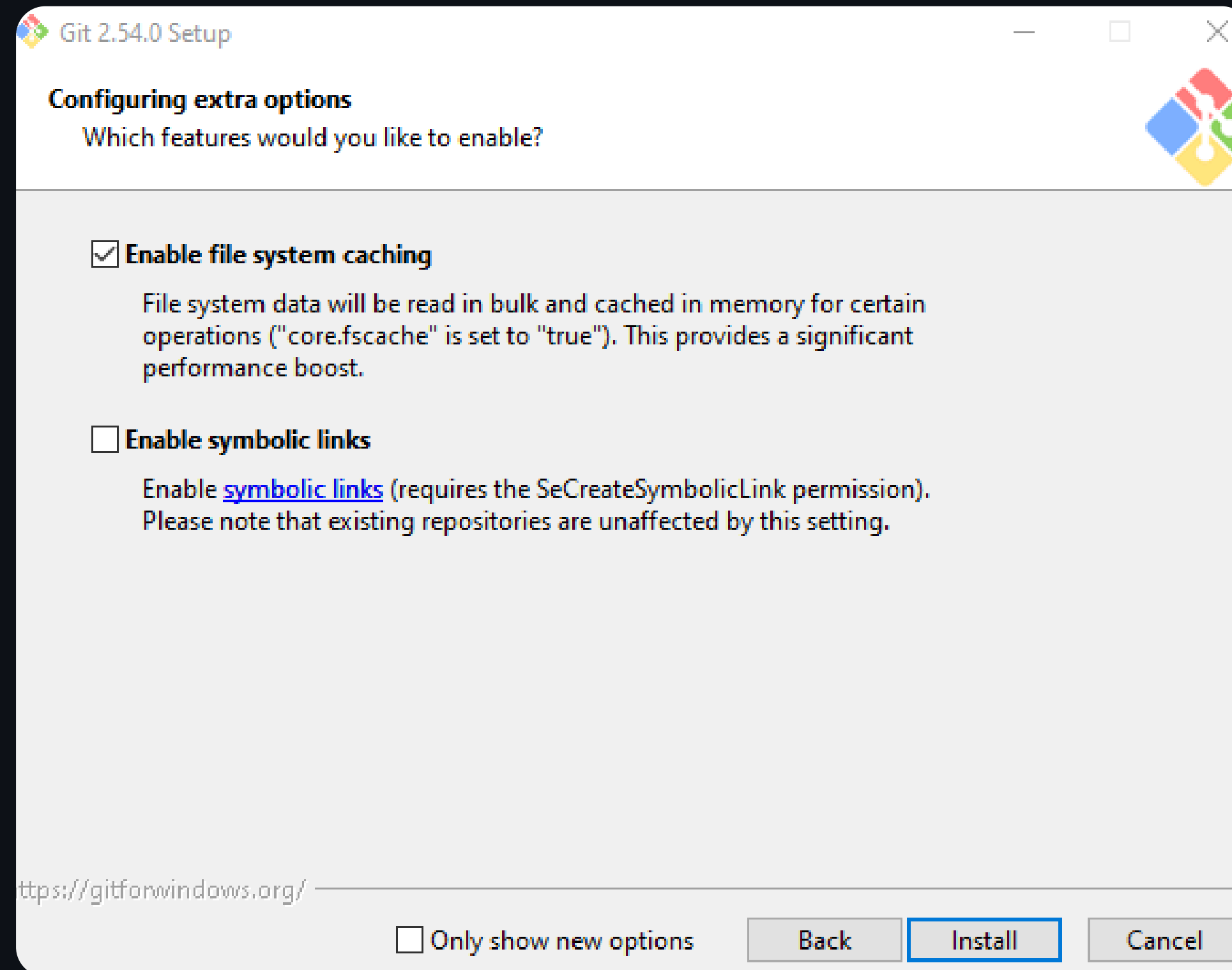










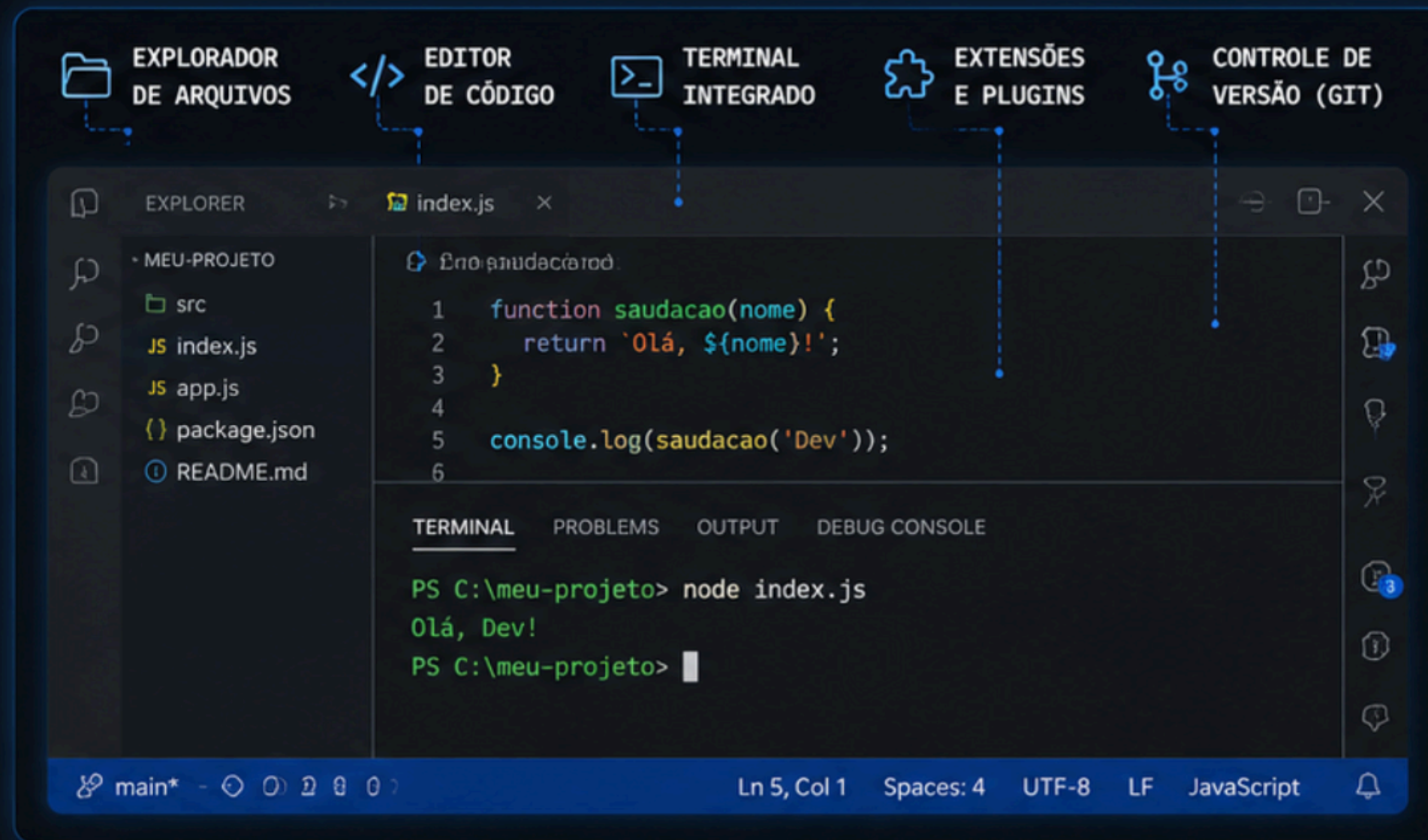


</> O QUE É UM IDE?

IDE (Integrated Development Environment) é um Ambiente de Desenvolvimento Integrado que reúne em um só lugar todas as ferramentas que um desenvolvedor precisa para escrever, testar, depurar e gerenciar código.

⚙️ PRINCIPAIS RECURSOS DE UM IDE

- ✓ Edição de código com destaque de sintaxe e autocompletar
- ✓ Navegação inteligente entre arquivos e funções
- ✓ Execução de código e depuração (breakpoints, logs, etc.)
- ✓ Terminal integrado
- ✓ Controle de versão (Git/GitHub)
- ✓ Extensões e plugins para aumentar a produtividade
- ✓ Gerenciamento de projetos



⚖️ IDE vs EDITOR DE TEXTO

EDITOR DE TEXTO

Focado apenas na edição de arquivos de texto.

VS

IDE

Ambiente completo para desenvolvimento de software.

★ EXEMPLOS DE IDEs POPULARES



VS Code



IntelliJ IDEA



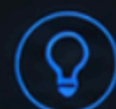
PyCharm



Eclipse

🏆 BENEFÍCIOS

- ✓ Mais produtividade e organização
- ✓ Menos erros e retrabalho
- ✓ Tudo integrado em um só lugar
- ✓ Foco no que realmente importa: código!



O IDE é o seu **centro de comando** para transformar ideias em soluções de software.



COMO FUNCIONA O TERMINAL INTEGRADO DO VSCODE

Uma interface de linha de comando integrada ao editor que permite executar comandos, automatizar tarefas e gerenciar projetos sem sair do ambiente de desenvolvimento.



PRINCIPAIS FUNÇÕES



Executar comandos Git (clone, commit, push, pull)



Navegar entre pastas e arquivos do projeto



Rodar aplicações, scripts e servidores locais



Instalar dependências (npm, Maven, pip, etc.)



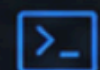
Automatizar processos de desenvolvimento

```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE
+ v | [icon] [trash] ... x

PS C:\meu-projeto> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   src/app.js

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\meu-projeto> █
```



COMANDOS BÁSICOS

<code>pwd</code>	→ mostra diretório atual
<code>ls</code>	→ lista arquivos
<code>cd nome-da-pasta</code>	→ entra em diretório
<code>mkdir nome</code>	→ cria pasta
<code>git status</code>	→ verifica alterações
<code>clear</code>	→ limpa terminal



ATALHOS ÚTEIS

<code>Ctrl + `</code>	→ abrir terminal
<code>Ctrl + C</code>	→ interromper processo
<code>Tab</code>	→ autocompletar comandos



BENEFÍCIOS

- ✓ Mais produtividade
- ✓ Menos troca de windows
- ✓ Controle direto do projeto
- ✓ Integração com Git/GitHub



O TERMINAL DO VSCODE TRANSFORMA SEU EDITOR EM UM AMBIENTE COMPLETO DE DESENVOLVIMENTO PROFISSIONAL.

Comandos básicos de gerenciamento de arquivos e pastas

Encontro 1

\$ mkdir meu-projeto

Criar uma pasta nova

Cria uma nova pasta vazia com o nome que você escolher. É o primeiro passo para organizar seu projeto.

\$ cd meu-projeto

Entrar em uma pasta

Navega para dentro de uma pasta. "cd" vem de change directory – mudar de diretório. Use cd .. para voltar.

\$ ls

Listar arquivos e pastas

Mostra tudo que existe dentro da pasta atual. Assim você sabe o que tem no lugar onde está no terminal.

\$ pwd

Onde estou agora?

Mostra o caminho completo da pasta em que você está. "pwd" vem de print working directory.

\$ clear

Limpa o terminal

mkdir

cd

ls

pwd

clear



CONFIGURANDO O GIT: USER.NAME E USER.EMAIL

Essas configurações identificam você como autor dos commits no GitHub.
São aplicadas globalmente (em todos os repositórios do seu computador).



POR QUE É IMPORTANTE?

- ✓ Seus commits terão seu nome e e-mail no histórico
- ✓ Facilita a colaboração em projetos
- ✓ Associa corretamente suas contribuições no GitHub
- ✓ Padrão profissional de identificação

1 ABRIR O TERMINAL



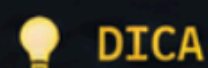
Abra o terminal integrado
do VSCode

Ctrl + `

2 CONFIGURAR O NOME DE USUÁRIO

Define o nome que aparecerá
como autor dos commits.

```
$ git config --global  
user.name "Seu Nome"
```



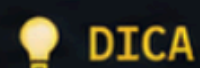
DICA

Use seu nome real ou
como deseja que apareça
nos commits.

3 CONFIGURAR O E-MAIL

Define o e-mail que aparecerá
como autor dos commits.

```
$ git config --global  
user.email "seu@email.com"
```



DICA

Use o mesmo e-mail
cadastrado na sua conta
do GitHub.

4 VERIFICAR SE FUNCIONOU

Lista as configurações atuais
do Git.

```
$ git config --list
```

Exemplo de saída:

```
user.name=Seu Nome  
user.email=seu@email.com  
core.autocrlf=true  
core.editor=code --wait  
...
```

5 VERIFICAR CAMPOS ESPECÍFICOS

Opcional: verifique cada campo
individualmente.

Nome:

```
$ git config user.name
```

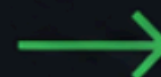
E-mail:

```
$ git config user.email
```



TUDO CERTO QUANDO...

Os comandos acima retornarem seu nome e e-mail corretamente.



PRONTO!

Agora seus commits estarão identificados corretamente no GitHub.



COMO SEU NOME E E-MAIL APARECEM NOS REGISTROS

As configurações de user.name e user.email são usadas pelo Git para identificar o autor de cada commit que você realiza.

EXEMPLO NA PRÁTICA

- 1 Você configura seu nome e e-mail

```
$ git config --global user.name "Seu Nome"
$ git config --global user.email "seu@email.com"
```

- 2 Faz um commit normalmente

```
$ git add .
$ git commit -m "Adiciona funcionalidade de login"
```

- 3 Seu nome e e-mail aparecem no commit!

O Git registra essas informações como autor da alteração.



BENEFÍCIOS

- ✓ Identifica corretamente o autor
- ✓ Mantém o histórico organizado
- ✓ Facilita a colaboração em equipe
- ✓ Padrão profissional no GitHub

EXEMPLO NO TERMINAL (git log)

Terminal

```
$ git log --oneline
a1b2c3d Adiciona funcionalidade de login (HEAD -> main)
Seu Nome <seu@email.com> Mon Apr 27 22:14:32 2026 -0300

d4e5f6g Corrige tela de cadastro
Seu Nome <seu@email.com> Sun Apr 12 16:49:10 2026 -0300

f7g8h9i Ajusta validações de formulário
Seu Nome <seu@email.com> Mon Apr 6 22:38:49 2026 -0300
~
```

Seu nome e e-mail aparecem em todos os commits que você realiza.



EXEMPLO NO GITHUB

Commits on Apr 27, 2026

Adiciona funcionalidade de login

Seu Nome committed 2 weeks ago

Corrige tela de cadastro

Seu Nome committed 2 weeks ago

No GitHub, seu nome aparece ao lado de cada commit enviado para o repositório.



DICA

Use sempre o mesmo e-mail do GitHub para associar seus commits ao seu perfil corretamente.

</> Configurar o GitHub no VSCode via terminal Git

1. Abrir o terminal integrado do VSCode

Menu: Terminal → New Terminal (ou Ctrl + `)

2. Configurar nome de usuário

Aparece em todos os commits

```
$ git config --global user.name "Seu Nome"
```

3. Configurar e-mail

Use o mesmo e-mail cadastrado no GitHub

```
$ git config --global user.email "seu@email.com"
```

4. Verificar se funcionou

user.name e user.email devem aparecer na lista

```
$ git config --list
```

5. Fazer login no GitHub (via browser)

Abre o GitHub no navegador para autenticar. Alternativa: token HTTPS

```
$ gh auth login
```



Baixar projeto de exemplo

Prática coletiva

1

Instrutor compartilha a URL do repositório de exemplo

Acessar o github pelo navegador

2

Pesquisar Pefil

Pesquisar o perfil "nome" na barra de busca do github

3

Acessar perfil e o repositório indicado

Entre no perfil e acesse o repositório

4

Baixar o projeto localmente

Baixe o projeto no seu computador

5

Acesse o projeto usando sua IDE (Ambiente de desenvolvimento)

Use sua IDE para abrir o projeto

Repositório local — Comandos essenciais

Use no dia a dia para controlar suas mudanças e manter o histórico do seu projeto.

COMANDO		O QUE FAZ	QUANDO USAR	EXEMPLO	OBSERVAÇÕES
<code>git init</code>		Inicializa o repositório Cria um novo repositório Git na pasta atual, gerando a pasta oculta <code>.git</code> onde o Git armazenará todo o histórico.	- Quando você está começando um novo projeto. - Quando deseja versionar uma pasta existente.	<pre># inicia um novo repositório git init</pre>	 Execute na raiz do projeto (na pasta principal).
<code>git status</code>		Estado atual dos arquivos Mostra quais arquivos foram modificados, adicionados à área de stage ou estão prontos para commit.	- Para verificar o que mudou desde o último commit. - Para conferir o que será incluído no próximo commit.	<pre># verifica o estado dos arquivos git status</pre>	 Sempre execute antes de fazer um commit para revisar as mudanças.
<code>git add .</code>		Prepara todos para commit Adiciona todos os arquivos modificados para a área de stage (preparação).	- Após fazer alterações nos arquivos do projeto. - Quando deseja incluir todas as mudanças de uma vez.	<pre># adiciona todos os arquivos modificados git add .</pre>  <u>Dica:</u> para adicionar arquivos específicos, use: <code>git add nome_do_arquivo</code>	 Use <code>.</code> (ponto) para adicionar todos os arquivos da pasta e subpastas.
<code>git commit -m "msg"</code>		Cria o commit (foto) Registra as mudanças que estão na área de stage no histórico do repositório, junto com uma mensagem descritiva.	- Após adicionar as mudanças com <code>git add</code>. - Para salvar um “ponto de controle” do projeto.	<pre># cria um commit com mensagem descritiva git commit -m "Correção do botão de login"</pre>  A mensagem deve ser clara e objetiva. Explique o que foi feito.	 Commits pequenos e frequentes são recomendados para manter um histórico organizado.
<code>git log .</code>		Histórico compacto Exibe o histórico de commits de forma resumida (uma linha por commit), mostrando o id curto e a mensagem.	- Para visualizar rapidamente o histórico do projeto. - Para entender a sequência de commits realizados.	<pre># mostra o histórico de commits de forma resumida git log .</pre> <pre># exemplo de saída a1b2c3d Correção do botão de login d4e5f6g Adição da página de perfil 7h8i9j0 Inicial do projeto</pre>	 Útil para ter uma visão geral rápida do histórico. Para ver detalhes completos, use <code>git log</code> .



BOA PRÁTICA: Faça commits frequentes e com mensagens claras. Isso facilita entender o histórico do projeto e reverter mudanças se necessário.

Inicializar um projeto — comandos básicos

```
$ mkdir meu-projeto
```

Criar a pasta do projeto

Cria uma nova pasta vazia para o seu projeto.

```
$ cd meu-projeto
```

Entrar na pasta

Navegar para dentro da pasta criada no terminal.

```
$ git init
```

Inicializar o repositório Git

Cria a pasta oculta `.git/` — onde o Git guarda todo o histórico. Agora o Git enxerga o projeto.

```
$ git status
```

Ver o estado atual

Mostra quais arquivos foram criados, modificados ou preparados. Use sempre que quiser saber o que está acontecendo.

O ciclo básico: editar → preparar → confirmar

Arquivo modificado ----> Com (git add .) você seleciona todos os arquivos modificados para commit ----> Com git commit você registra no histórico

Diretório de trabalho

Arquivo criado ou modificado. Git sabe que mudou, mas ainda não vai salvar.

git add



Área de preparação

Preparado com git add. Está na fila para o próximo commit.

git
commit



Repositório

Confirmado com git commit. Faz parte do histórico para sempre.

```
$ git add arquivo.txt
```

— preparar arquivo específico

```
$ git commit -m "mensagem"
```

— salva o estado no histórico

```
$ git add .
```

— preparar todos os modificados

```
$ git log
```

— ver histórico compacto

Repositório remoto — Comandos essenciais

Use estes comandos para conectar seu repositório local a um remoto e sincronizar alterações.

COMANDO		O QUE FAZ	QUANDO USAR	EXEMPLO	OBSERVAÇÕES
<code>git clone <url></code>		Baixa um repositório remoto Cria uma cópia completa de um repositório remoto na sua máquina, incluindo todo o histórico de commits e branches.	- Quando você quer obter uma cópia de um projeto já existente no GitHub ou em outro servidor. - Para começar a trabalhar em um projeto existente.	<pre># clona um repositório do Github git clone https://github.com/usuario/projeto.git</pre>	 Após clonar, use <code>git remote -v</code> para verificar as conexões remotas.
<code>git remote add origin <url></code>		Vincula o repositório local a um remoto Adiciona um novo repositório remoto com o nome "origin" (convencional) e sua respectiva URL.	- Quando você já tem um repositório local e quer conectá-lo a um remoto. - Geralmente usado apenas uma vez por repositório.	<pre># conecta o remoto ao seu repositório local git remote add origin https://github.com/usuario/projeto.git</pre>	 O nome "origin" é apenas uma convenção. Você pode usar outro nome se quiser.
<code>git remote -v</code>		Lista os repositórios remotos Exibe os remotos configurados e suas URLs, mostrando também qual é o endereço para fetch (buscar) e push (enviar).	- Para verificar se o remoto foi configurado corretamente. - Para checar ou consultar as URLs dos remotos existentes.	<pre># lista todos os remotos configurados git remote -v origin https://github.com/usuario/projeto.git (fetch) origin https://github.com/usuario/projeto.git (push)</pre>	 Você verá algo como ao lado, com as URLs usadas para buscar e enviar dados do remoto.
<code>git push -u origin main</code>		Envia suas alterações para o repositório remoto Envia os commits da branch atual (ex.: main) para o remoto. A opção -u configura o rastreamento da branch local com a remota.	- Após fazer commits e querer enviar suas mudanças para o remoto. - Na primeira vez que você envia uma branch para o remoto, use -u.	<pre># envia os commits da branch main e define rastreamento git push -u origin main</pre>	 Depois da primeira vez, você pode usar apenas <code>git push</code> na mesma branch.



FLUXO TÍPICO COM REMOTO



1. Conectar remoto
`git remote add origin <url>`



2. (Opcional) Clonar
`git clone <url>`



3. Fazer alterações e commits
`git commit -m "msg"`



4. Enviar para o remoto
`git push -u origin main` (primeira vez)



5. Manter sincronizado
`git pull` (para atualizar)



Envio do projeto local para o GitHub

1

Iniciar o git na pasta, adicione o arquivo e crie o primeiro commit

`git init` | `git add .` | `git commit -m "primeiro commit"`

2

Criar e Vincular o repositório remoto

Diz ao Git que "origin" é a URL do GitHub:

```
$ git remote add origin https://github.com/usuario/meu-projeto.git
```

3

Verificar se foi vinculado

Lista os repositórios remotos configurados:

```
$ git remote -v
```

4

Enviar pela primeira vez

-u salva a configuração; nas próximas vezes basta git push:

```
$ git push -u origin main
```

5

Conferir no GitHub

Acessar github.com/seu-usuario/meu-projeto — commits e arquivos já visíveis!

Resumo dos comandos — Encontro 1

Configuração inicial

```
git --version
```

verifica instalação

```
git config --global user.name  
"Nome"
```

define o nome

```
git config --global user.email  
"email"
```

define o e-mail

```
git config --list
```

lista configurações

Repositório local

```
git init
```

inicializa o repositório

```
git status
```

estado atual dos arquivos

```
git add .
```

prepara todos para commit

```
git commit -m "msg"
```

cria o commit (foto)

```
git log --oneline
```

histórico compacto

Repositório remoto

```
git clone <url>
```

baixa do GitHub

```
git remote add origin <url>
```

vincula ao GitHub

```
git remote -v
```

lista remotos

```
git push -u origin main
```

envia pela 1ª vez

```
git commit -m "Encontro 1 concluído!"
```

Controle de versão com Git — visão simples

Explicação no quadro — analogia da linha do tempo de fotos



Cada commit é uma foto salva do projeto. Você pode voltar para qualquer foto a qualquer momento — sem perder nada.

commit

= registra a modificação no histórico

repositório

= Onde fica o projeto

histórico

= Todos os registros de modificação

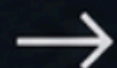
</> O QUE É CONTROLE DE VERSÃO?

É um sistema que **registra todas as mudanças** feitas nos arquivos do seu projeto ao longo do tempo, permitindo voltar atrás, comparar e trabalhar em equipe.

COMO FUNCIONA (DE FORMA SIMPLES)



1. Você cria ou edita um arquivo.



2. Você salva as mudanças (commit).



3. O sistema registra essa mudança no histórico.



4. Você pode voltar, comparar ou continuar de onde parou.



5. Outras pessoas podem fazer o mesmo e tudo fica sincronizado.

POR QUE É IMPORTANTE?



SEGURANÇA

Suas alterações ficam salvas. Você pode voltar atrás se algo der errado.



HISTÓRICO

Veja tudo que foi feito, quando e por quem.



COLABORAÇÃO

Várias pessoas podem trabalhar juntas no mesmo projeto.



COMPARAÇÃO

Compare versões e veja o que mudou entre elas.



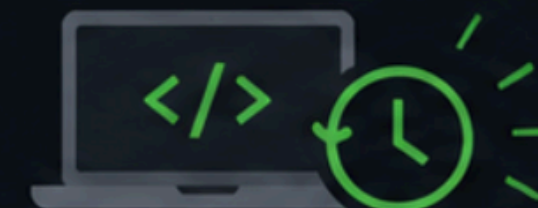
ORGANIZAÇÃO

Mantenha o projeto organizado e fácil de gerenciar.

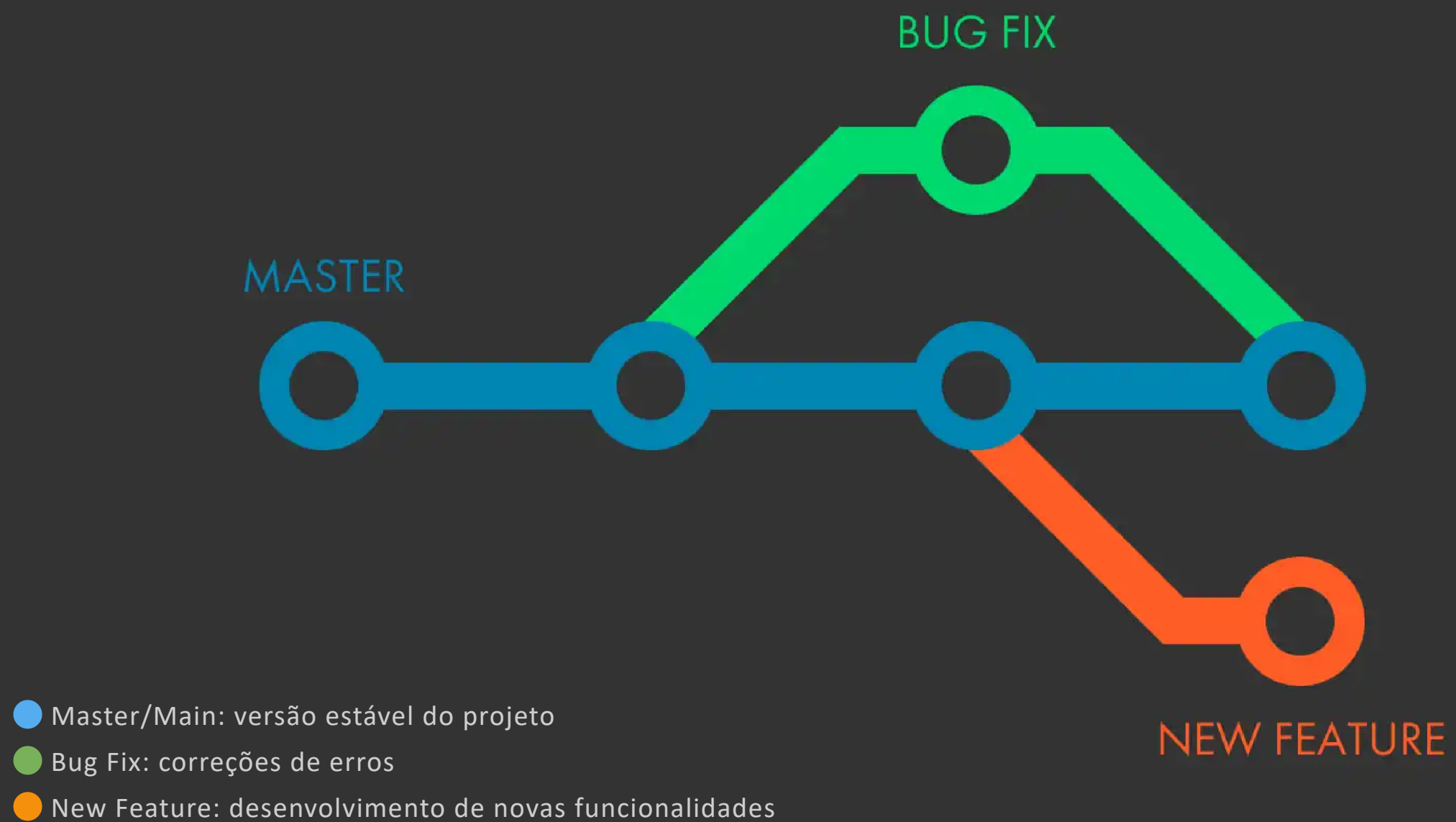


Controle de versão é como ter uma máquina do tempo do seu projeto.

Você pode salvar, comparar e evoluir com segurança!



ESTRATÉGIA DE VERSIONAMENTO COM GIT



EXEMPLO DE HISTÓRICO DE COMMITS EM TERMINAL

```
Terminal Local x + v
2
commit 9fae5bbfac28b3f020cbb440da3a9ef69c184f58 (HEAD -> main, origin/main)
Author: Haelyton Maicon <haelyton@nexuscloud.com.br>
Date: Mon Apr 27 22:14:32 2026 -0300

    Logica Beecrowd e inicio sistema estacionamento

commit 4ca42499c59a59d9cbffc96dcaed512fbb62aca0
Author: Haelyton Maicon <haelyton@nexuscloud.com.br>
Date: Sun Apr 12 16:49:10 2026 -0300

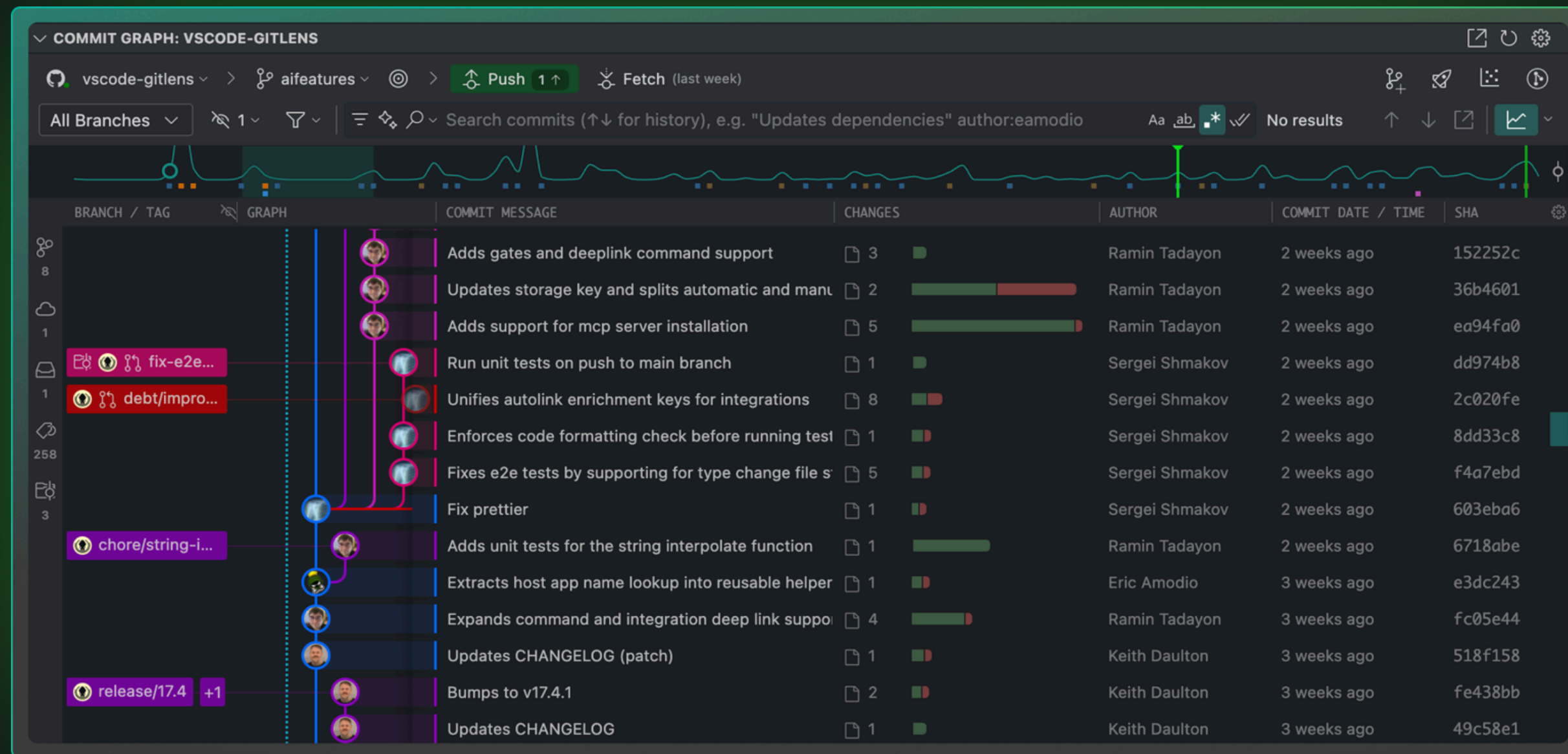
    Estudos Logicas

commit 2019a344b68ddf437a6e086764d57ced159a7e1a
Author: Haelyton Maicon <haelyton@nexuscloud.com.br>
Date: Mon Apr 6 22:30:49 2026 -0300

    Lógicas já implementadas
~
~
~
~
~
~
~
~
26 (END)
```

- Commit Hash: identificador único da alteração
- 👤 Author: responsável pela modificação.
- 📅 Date: data da alteração
- ✍ Message: descrição da mudança realizada

ESTRATÉGIA DE VERSIONAMENTO COM GIT





ENCONTRO 1 FINALIZADO!

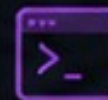
Parabéns por concluir o primeiro encontro!
Você deu o primeiro passo para dominar o Git e o GitHub.



Entendemos o
que é **Git** e **GitHub**



Exploramos **branches**
e versionamento



Configuramos o
ambiente no **VSCode**



Configuramos usuário
e vimos **commits**



PRÓXIMO ENCONTRO: ENCONTRO 2

Vamos avançar ainda mais!

**HISTÓRICO, NAVEGAÇÃO
E DESFAZIMENTO**



Continue praticando e explorando!
O conhecimento te leva mais longe.